

Rebuilding isProg

A Step-by-Step Verification that isProg is WHILE-Computable

Arif Akbarul Huda

Theory of Computation

Objective

- Prove that the function

$$\text{isProg} : \mathbb{N} \rightarrow \mathbb{N}$$

is WHILE-computable.

- $\text{isProg}(x_1)$ returns 1 if x_1 encodes a valid GOTO program, else 0.
- Approach: deconstruct the program encoding step by step using only WHILE constructs and already-proved helper functions.

Encoding of a GOTO Program

A GOTO program with k inputs, maximum variable index m , and n labelled statements

$$L_1 : S_1; L_2 : S_2; \dots L_n : S_n$$

is encoded as the list

$$(k, m, \text{enc}(S_1), \text{enc}(S_2), \dots, \text{enc}(S_n))$$

and then turned into a single natural number using the list encoding

$$\text{enc}((a_1, \dots, a_\ell)) = \pi(a_1 + 1, \text{enc}((a_2, \dots, a_\ell))), \quad \text{enc}(()) = 0.$$

Preliminaries – Helpers We Already Have

All the following are WHILE-computable (lectures 1–4):

- $\text{isList}(x)$ – returns 1 if x is a valid list encoding.
- $\text{len}(x)$ – length of the list.
- $\text{fst}(x)$ – first element of a pair / head of a list.
- $\text{snd}(x)$ – second element of a pair / tail of a list.
- Basic arithmetic: $x - 1$ (predecessor), $x + 1$ (successor), zero test.

Key convention: Every list element is stored as $a + 1$; empty list is 0.
Hence we always subtract 1 after fst .

Checkpoints Overview

We verify `isProg` in 9 sequential checkpoints:

- 1 Check `x1` is a valid list with `len` ≥ 2 .
- 2 Extract `k` and `m`, verify the remainder is a list.
- 3 Compute `n` (number of statements) from the tail.
- 4 Set up the loop over statements (countdown `n`).
- 5 For each statement: fetch `enc(Si)` and check it is a list.
- 6 Decode the statement type and parameter list; initialise flag.
- 7 Dispatch on the type (flat guarded blocks) – check length and bounds.
- 8 Advance to the next statement (`snd` + decrement counter).
- 9 After the loop, output the result contained in `x0`.

Checkpoint 1 – Is the Input a Valid Program List?

Focus: $(k, m, \text{enc}(\mathbf{S}_1), \dots, \text{enc}(\mathbf{S}_n))$

```
x2 := isList(x1);  
IF x2 = 0 THEN x0 := 0; HALT END;
```

```
x3 := len(x1);  
IF x3 < 2 THEN x0 := 0; HALT END;
```

- The outer encoding must be a proper list.
- It must contain at least k and m , so length ≥ 2 .

Checkpoint 2 – Extract k and m

Focus: $(k, m, \text{enc}(S_1), \dots, \text{enc}(S_n))$

```
x4 := fst(x1);  
x4 := x4 - 1;           // k = number of inputs  
  
x5 := snd(x1);          // (m+1, enc(S1), ...)  
IF isList(x5) = 0 THEN x0 := 0; HALT END;  
  
x6 := fst(x5);  
x6 := x6 - 1;           // m = max variable index  
x7 := snd(x5);          // list of statements
```

- `fst` then subtract 1 recovers the true value (the +1 encoding).
- The remainder after k must still be a well-formed list.

Checkpoints 3 & 4 – Statement Count and Loop Setup

Focus: $(k, m, \text{enc}(\mathbf{S}_1), \dots, \text{enc}(\mathbf{S}_n))$ **CP3** – Number of statements:

```
x8 := len(x7);           // n
```

CP4 – Initialise loop variables (countdown from n):

```
x9 := x7;                // pointer into statement list
x10 := x8;                // remaining iterations
x0 := 1;                  // assume valid
```

- x9 will be advanced with `snd`, x10 decremented each round.

Checkpoints 5 & 6 – Fetch and Decode a Statement

Focus: $(k, m, \dots, \text{enc}(\mathbf{S}_i), \dots)$

Inside the loop (WHILE $x_{10} \neq 0$ DO ...):

```
x11 := fst(x9);           // enc(S_i)
x12 := isList(x11);
IF x12 = 0 THEN x0 := 0    // malformed
ELSE
x13 := fst(x11);
x13 := x13 - 1;           // type (1..5)
x14 := snd(x11);          // parameter list
x15 := 0;                 // valid-type flag
END
```

- The statement encoding must itself be a valid list.
- x_{15} will become 1 when a type block matches.

Checkpoint 7 – Type Dispatch (Guarded Blocks)

Focus: $(k, m, \dots, \text{enc}(\mathbf{S}_i), \dots)$

Each of the five types is checked by a flat guarded block.

Example: **Type 1** ($x_i := x_i + 1$)

```
IF x15 = 0 THEN
x16 := x13; x16 := x16 - 1;           // type-1
IF x16 <> 0 THEN (skip)
ELSE
x15 := 1;
// parameter length must be 1
x17 := len(x14);
x18 := x17; x18 := x18 - 1;
IF x18 <> 0 THEN x0 := 0 END;         // len > 1
IF x17 <> 0 THEN ELSE x0 := 0 END;    // len = 0
// variable i <= m ?
x19 := fst(x14); x19 := x19 - 1;
x20 := x19; x21 := x6;
WHILE x21 <> 0 DO x20 := x20 - 1; x21 := x21 - 1 END;
IF x20 <> 0 THEN x0 := 0 END          // i > m
END
END
```

Guard IF $x_{15}=0$ THEN ... prevents later blocks from firing after a match.

Checkpoint 7 – Types 2, 3, 4, 5 (Summary)

Focus: $(k, m, \dots, \text{enc}(\mathbf{S}_i), \dots)$

- **Type 2** (dec): same as type 1 (length = 1, $i \leq m$).
- **Type 3** (GOTO L_j): length = 1, label $j \leq n$ (uses x_8).
- **Type 4** (IF $x_i=0$ THEN GOTO L_j):
 - length = 2, extract i and j .
 - check $i \leq m$ and $j \leq n$ with two bound loops.
- **Type 5** (HALT): parameter must be the empty list 0.

After all 5 blocks:

```
IF x15 = 0 THEN x0 := 0 END    // unknown type
```

Comparison Table – Statement Type Checks

Type	Syntax	#Params	Meaning	Bound check
1	$x_i := x_i + 1$	1	variable i	$i \leq m$
2	$x_i := x_i - 1$	1	variable i	$i \leq m$
3	GOTO L_j	1	label j	$j \leq n$
4	IF $x_i = 0$ THEN GOTO L_j	2	variable i , label j	$i \leq m \wedge j \leq n$
5	HALT	0	(none)	(none)

All bound checks are performed by subtraction until zero (the WHILE loop pattern).

Checkpoints 8 & 9 – Advance and Final Result

Focus: Loop over all $\text{enc}(S_i)$ completed. **CP8** – move to next statement:

```
x9 := snd(x9);  
x10 := x10 - 1  
END    // end of WHILE loop
```

CP9 – after the loop x_0 holds the verdict.

- x_0 was set to 1 initially; any error changed it to 0.
- Because we never reset x_0 to 1, the correct answer persists.
- The procedure is total – every loop terminates.

$\Rightarrow$ `isProg` is WHILE-computable.